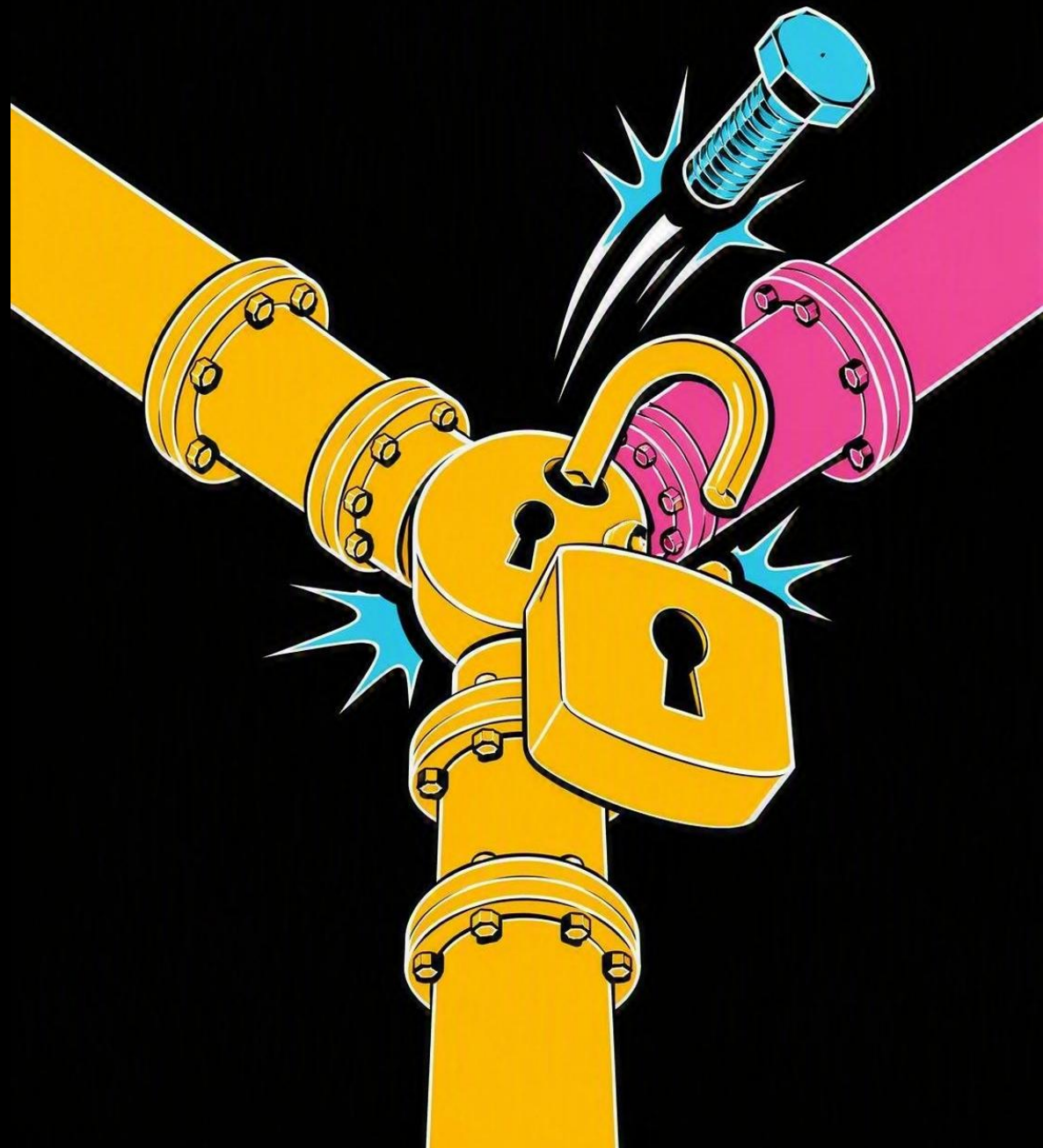
When Interoperability Breaks Security

Governance integration: Fabric · Snowflake · Databricks

Matthias Nohl | Andreas Rederer

Daten-WG 2026 · Audiothek · 60 minutes





Your speakers

Meet the roommates

On the right

Matthias Nohl

Strategic Partner Manager / Community Lead · b.telligent

- At b.telligent since 2015, Microsoft data platforms
- Diploma Medical Informatics
- Based in Munich, Bavaria

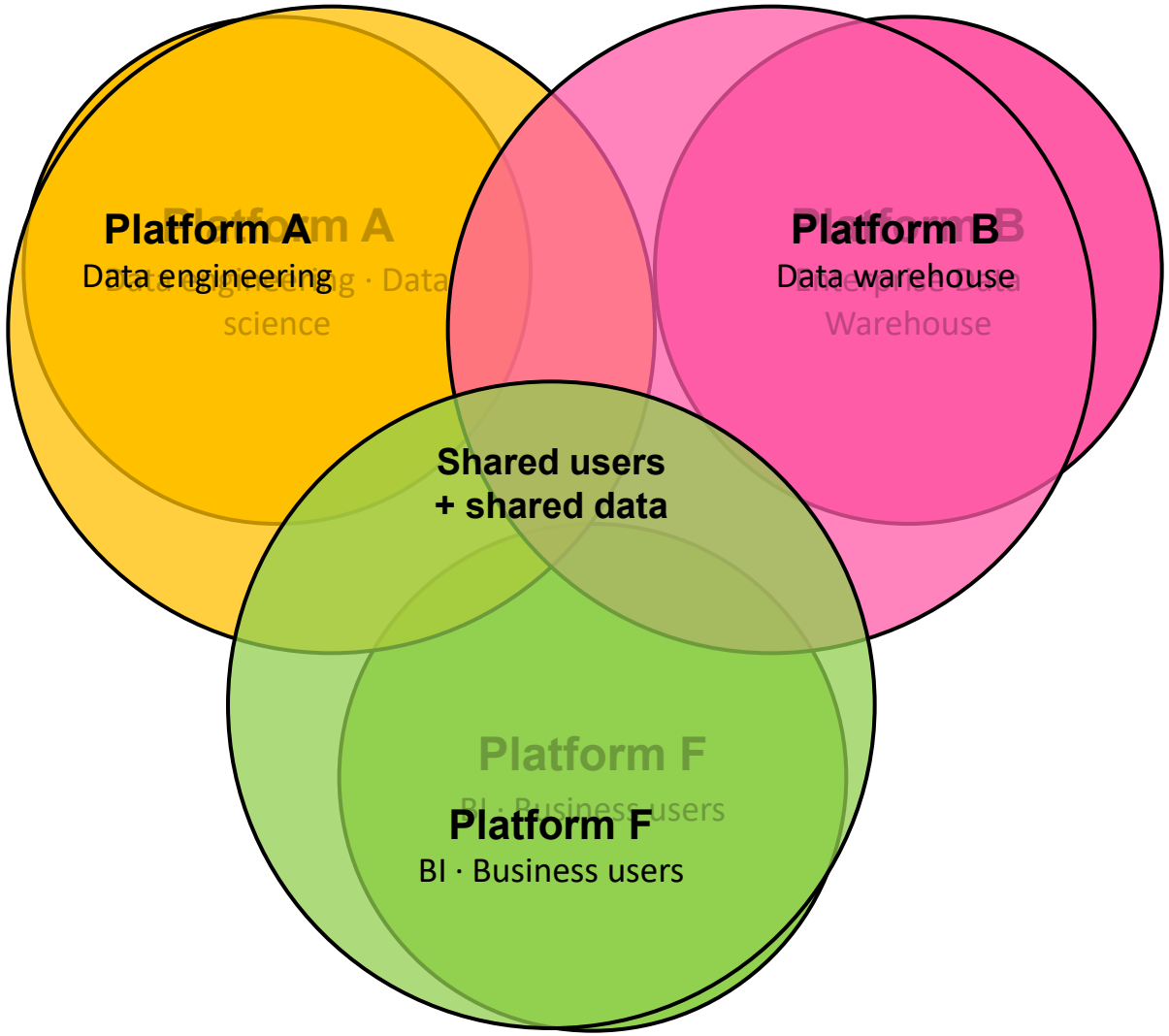
On the left

Andreas Rederer

Cloud + AI Solution Engineer - Data Platforms · Microsoft

- Co-author of Policy Weaver (github.com/microsoft/Policy-Weaver)
- M.Sc. Mathematics
- Based in Landshut, Bavaria

Enterprises often operate multiple data platforms



Typically: different user groups

Each team works on the platform that fits its job.

But what happens if they overlap?

The same people and the same tables show up on more than one platform.

The problem: each platform governs alone

Every engine has its own access rules.



The dream: define data access policies only once

Agenda

The Good, the Bad and the Ugly(?)

01

The Good

Modern platforms work more and more seamlessly together

- The interoperability of modern data platforms
- Integration options with Fabric

02

The Bad

Security policies are left behind

- How data access policies work
- Each platform has its own mechanisms
- Security policies typically don't travel

03

The Ugly(?)

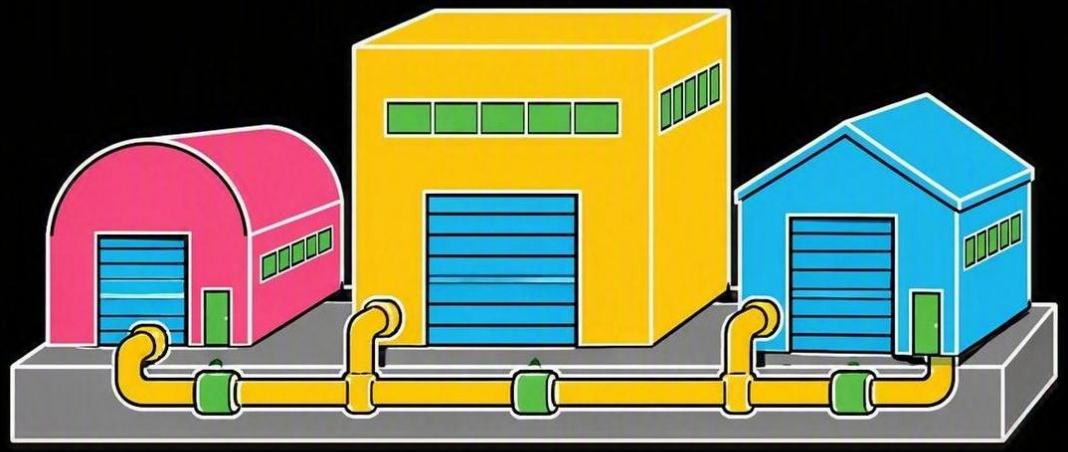
Policy Weaver bridges the gap

- Policy Weaver - Why, What, How
- Deep dive + Demo
- Limitations, recommendations, what's next

Part 1

The Good: modern platforms work more and more seamlessly together

01



Common ground

Modern engines meet in the data lake

QUERY ENGINES

Azure Databricks

Delta tables in ADLS Gen2 or OneLake

Snowflake

Iceberg tables on an external volume in OneLake or ADLS

Microsoft Fabric

Delta in OneLake

OPEN TABLE FORMATS

Delta | **Apache Iceberg** | Open specs: every engine can read them

LAKE STORAGE

ADLS Gen2 | **OneLake** | ADLS Gen2 API · shortcuts to ADLS, S3, GCS

One copy of the data, readable by every engine

Boundary Conditions

Integration options with Fabric

Copy

Paths 1 + 2

COVERS

Mirroring, Copy Job

CARRIED

Data + schema, replicated into OneLake as Delta

NOT CARRIED AUTOMATICALLY

Source roles and policies; the replica is a new Fabric item with its own roles

Zero-copy

Paths 3 + 4

COVERS

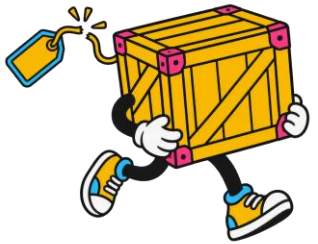
Iceberg shared format · shortcut · catalog mirroring · federation

CARRIED

Table state + schema or only an access path / reference

NOT CARRIED AUTOMATICALLY

No cross-platform authorization; source and target controls must line up



Copy: Snowflake Mirroring replicates tables without ETL pipelines



Snowflake → Fabric Mirroring

Fabric replicates tables into OneLake as Delta, near real-time via change tracking.

Fabric replication compute is free; Snowflake warehouse compute is billed.

OneLake · Mirrored database

The replica is a new Fabric item. Access starts from workspace roles and OneLake security, not from Snowflake.

snowflake_mirroring.sql — SQL

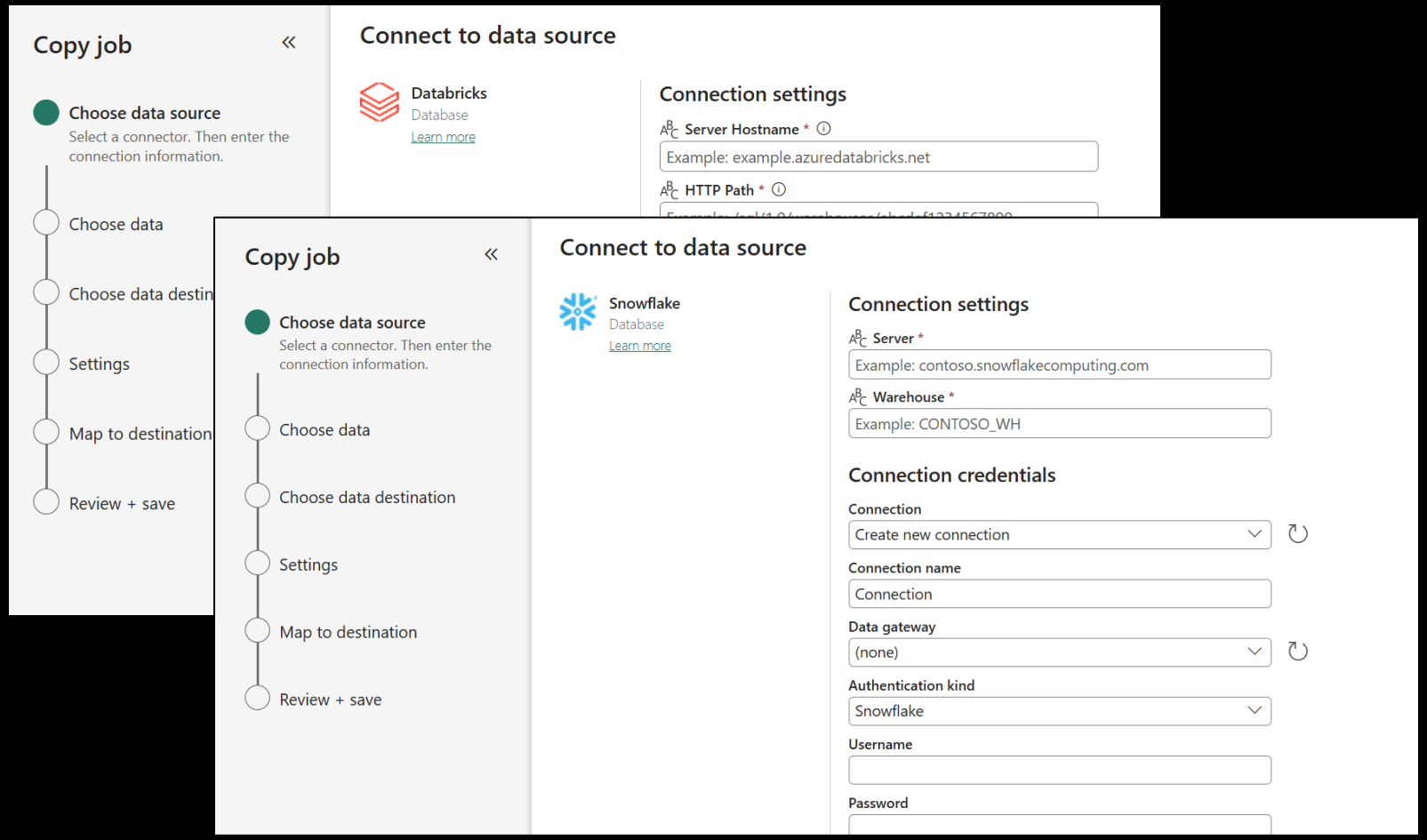
```sql
-- ▸ 1 · Read-only role for Fabric Mirroring
CREATE ROLE FABRIC_RO;
GRANT USAGE ON WAREHOUSE COMPUTE_WH TO ROLE FABRIC_RO;
GRANT USAGE ON DATABASE SALES TO ROLE FABRIC_RO;
GRANT USAGE ON SCHEMA SALES.PUBLIC TO ROLE FABRIC_RO;
GRANT SELECT ON ALL TABLES IN SCHEMA SALES.PUBLIC TO ROLE FABRIC_RO;
GRANT SELECT ON FUTURE TABLES IN SCHEMA SALES.PUBLIC TO ROLE FABRIC_RO;
GRANT CREATE STREAM ON SCHEMA SALES.PUBLIC TO ROLE FABRIC_RO;

-- ▸ 2 · Technical user (key pair instead of password)
CREATE USER FABRIC_MIRROR TYPE = SERVICE
  DEFAULT_ROLE = FABRIC_RO DEFAULT_WAREHOUSE = COMPUTE_WH
  RSA_PUBLIC_KEY = '<public key>';
GRANT ROLE FABRIC_RO TO USER FABRIC_MIRROR;

-- ▸ 3 · Change tracking per mirrored table
ALTER TABLE SALES.PUBLIC.ORDERS SET CHANGE_TRACKING = TRUE;
SHOW GRANTS TO ROLE FABRIC_RO;   -- verify

-- ▸ 4 · Fabric
-- New item > Mirrored Snowflake > connection FABRIC_MIRROR
-- Select tables > Mirror · then: Manage OneLake data access
-- Data arrives; roles only via role replication (preview)
```

Copy: Copy Jobs land source tables in a Lakehouse on a schedule



Copy Job · Lakehouse

Set up a Fabric Lakehouse and land the data as Delta tables in it. Pipelines or notebooks work the same way.

Shared format: one copy of the files, read by several engines



Snowflake → Iceberg on OneLake

Snowflake writes Iceberg tables directly to an external volume in OneLake.

A shortcut exposes the folder; OneLake adds Delta metadata, so Fabric engines read it as Delta.

snowflake_iceberg.sql — SQL

```sql
-- Iceberg on OneLake: one copy, two engines
CREATE EXTERNAL VOLUME ONELAKE_VOL STORAGE_LOCATIONS = ((
  NAME = 'onelake' STORAGE_PROVIDER = 'AZURE'
  STORAGE_BASE_URL = 'azure://onelake.dfs.fabric.microsoft.com/...'
  AZURE_TENANT_ID = '<tenant-id>'));
CREATE ICEBERG TABLE SALES.PUBLIC.ORDERS_ICE
  CATALOG = 'SNOWFLAKE' EXTERNAL_VOLUME = 'ONELAKE_VOL'
  BASE_LOCATION = 'orders' AS SELECT * FROM SALES.PUBLIC.ORDERS;
-- Fabric: Lakehouse > New shortcut > Iceberg table folder
```

Databricks → Delta on Data Lake Storage → OneLake Shortcut

Databricks writes Delta tables to an external location in ADLS Gen2.

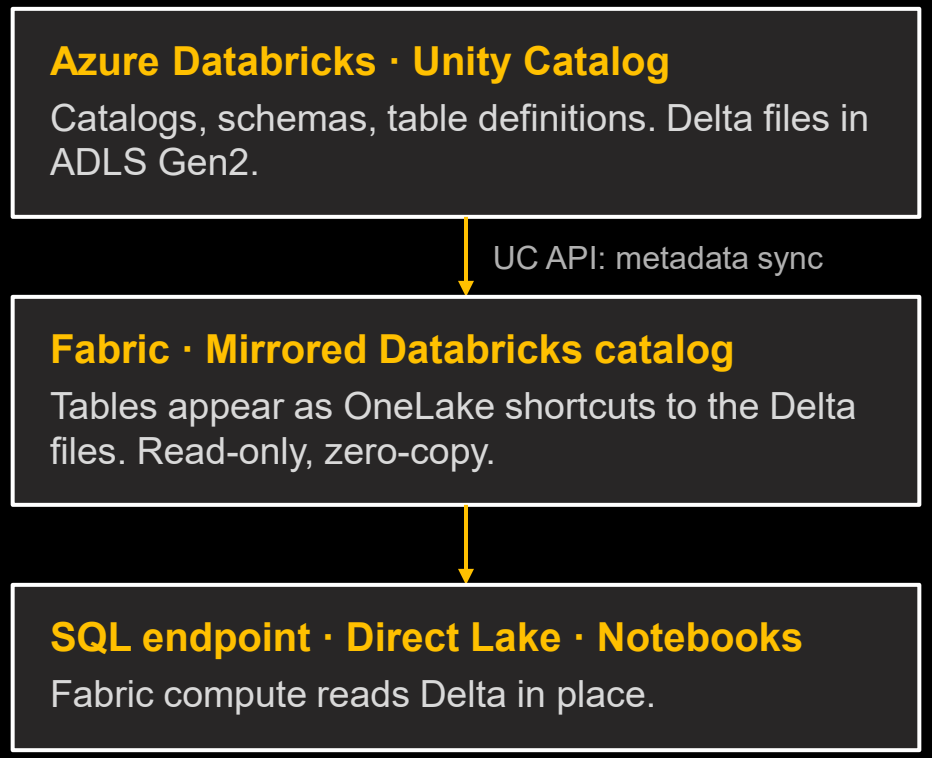
Fabric reads the same files via shortcut, exposed as Delta.

Snowflake → Iceberg on Data Lake Storage → OneLake Shortcut

Snowflake keeps Iceberg tables on its existing external volume in ADLS Gen2.

A OneLake shortcut points to that folder; Fabric reads it as Delta, no copy.

Reference: Fabric mirrors the metadata and creates OneLake Shortcuts



unity_catalog_setup.sql — SQL

```sql
-- 1) Metastore: allow external engines
--    Catalog Explorer > Metastore > External data access
-- 2) Privileges for the identity behind the Fabric connection
GRANT USE CATALOG ON CATALOG prod TO `fabric-mirror-sp`;
GRANT USE SCHEMA ON SCHEMA prod.sales TO `fabric-mirror-sp`;
GRANT EXTERNAL USE SCHEMA ON SCHEMA prod.sales
  TO `fabric-mirror-sp`;
GRANT SELECT ON SCHEMA prod.sales TO `fabric-mirror-sp`;

-- 3) Locked-down ADLS Gen2 (optional)
--    Networking > Resource instances > Microsoft.Fabric
--    IAM: Storage Blob Data Reader > workspace identity
-- 4) Fabric workspace (Contributor or higher)
--    New item > Mirrored Azure Databricks catalog
--    Connection: workspace URL + OAuth (user or SP)
--    Select catalog > schemas > tables > Create
-- Requires Fabric runtime with Spark 3.4 / Delta 2.4+

-- 5) Optional shared format: Iceberg reads via UniForm
ALTER TABLE prod.sales.orders SET TBLPROPERTIES (
  'delta.enableIcebergCompatV2' = 'true',
  'delta.universalFormat.enabledFormats' = 'iceberg');
```

Part 2

The Bad: security policies are left behind

02



Data is not everything, what about governance?

Yeah, Data Integration is super easy



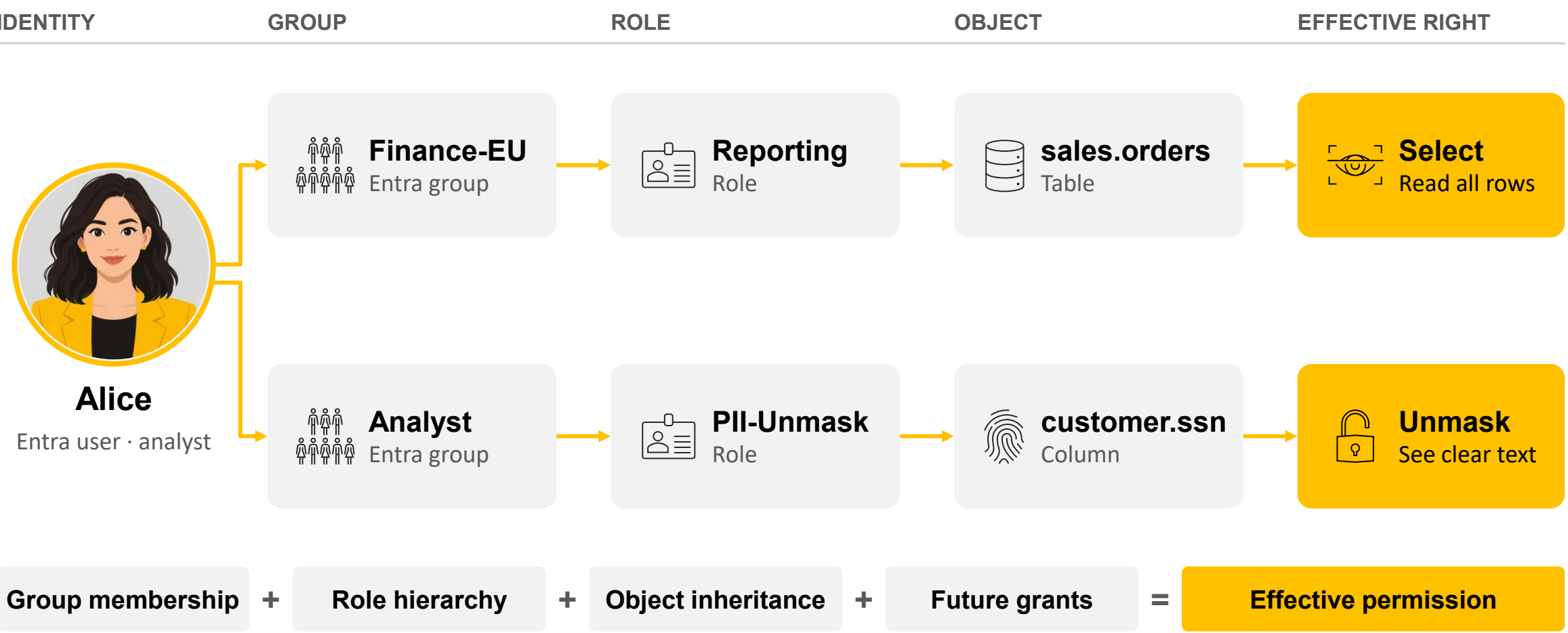
… but data access policies are left behind*



(*with one recently announced exception)

Role Expansion

Computing access means walking the graph



Catalog lineage shows where data flows. Read access follows only from the role graph above.

Three control layers that get confused

Lineage

Where does the data come from?
How was it transformed?
Which objects are related?

Authorization

Who may see which data?
Through which path?
Under which row/column restriction?

Audit

Which identity accessed what, and when?
Can logs be correlated across platforms?



Field observation: once lineage shows up in the catalog, teams assume the access question is settled.

Control Separation

Three platforms – three systems of authorization

QUERY ENGINES · WHERE THE QUERY RUNS

Snowflake

- **Virtual warehouses:** SQL
- **Snowpark:** Python · Java · Scala, executed in the warehouse

Azure Databricks

- **Spark:** clusters, jobs, notebooks
- **Photon:** vectorized engine for Spark and SQL
- **Databricks SQL:** serverless SQL warehouses

Microsoft Fabric

- **Spark:** Lakehouse, notebooks
- **MPP SQL:** SQL endpoint · Warehouse
- **VertiPaq:** Power BI · Direct Lake
- **Eventhouse:** KQL

AUTHORIZATION GATE · EACH PLATFORM DECIDES ALONE

Snowflake Horizon

- **RBAC:** roles, grants, role hierarchy
- **Masking policies:** column level
- **Row access policies:** row level
- **Tag-based masking:** policy follows the tag

→ **Enforced in the virtual warehouse at query time**

Unity Catalog

- **Privileges:** catalog · schema · table
- **Row filters:** row level
- **Column masks:** column level
- **ABAC:** policies on governed tags (GA)

→ **Enforced in Databricks compute; external engines via Iceberg REST scan planning (Beta)**

OneLake security

- **Workspace/item roles:** control plane
- **OneLake roles:** table/folder access (data plane)
- **RLS · CLS:** row and column level per role

→ **Enforced at query time in the reading engine: Spark, SQL endpoint, Direct Lake**

Security Model Comparison

Grants do not map 1:1 between the three platforms

Dimension	Snowflake	Databricks	Fabric / OneLake
Primary securables	Account · DB · Schema · Table	Metastore · Catalog · Schema · Table	Workspace · Item · Schema · Folder/Table
Principals	User · Role (account and database roles)	User · Group · Service Principal	User · Group · Service Principal
Inheritance	Role hierarchy: roles granted to roles	Hierarchical securables / privileges	Workspace/item permissions + OneLake roles
Granularity	Row access / masking policies	Row filters / column masks	Folder/Table + RLS/CLS
Effective access	Primary + secondary roles, incl. inherited roles	Privileges + ownership + inheritance	Control plane + data plane; Admin, Member, Contributor bypass RLS/CLS



Equivalence cannot be copied object by object. The systems look similar but differ in the details.

Policy gap

Permissions have to be rebuilt in Fabric on every path

Data path: solved by every pattern

Source tables

Unity Catalog · Snowflake

→

Four paths

Mirroring · Copy Job · shared format · catalog mirroring

→

Fabric consumption

SQL endpoint · Direct Lake · BI

Policy path: carried by no pattern*

Source grants

UC grants, Snowflake roles, masks and row policies

⇢

Missing link

Nothing translates source grants into OneLake roles

⇢

OneLake security

Roles must match the source per user and table

* Exception: Snowflake Mirroring can replicate roles into OneLake security (public preview since FabCon Europe 2026).

Failure Modes

Policy translation fails at six points

F1 · Policy left behind

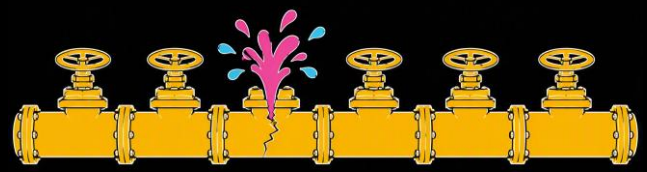
Mirroring copies the data but not its policy

F2 · Semantic mismatch

The same privilege name grants different rights

F3 · Shadow access

Alternative path bypasses the source control



F4 · Default broad access

Workspace roles from Contributor up bypass RLS/CLS

F5 · Identity mismatch

Principal cannot be resolved unambiguously

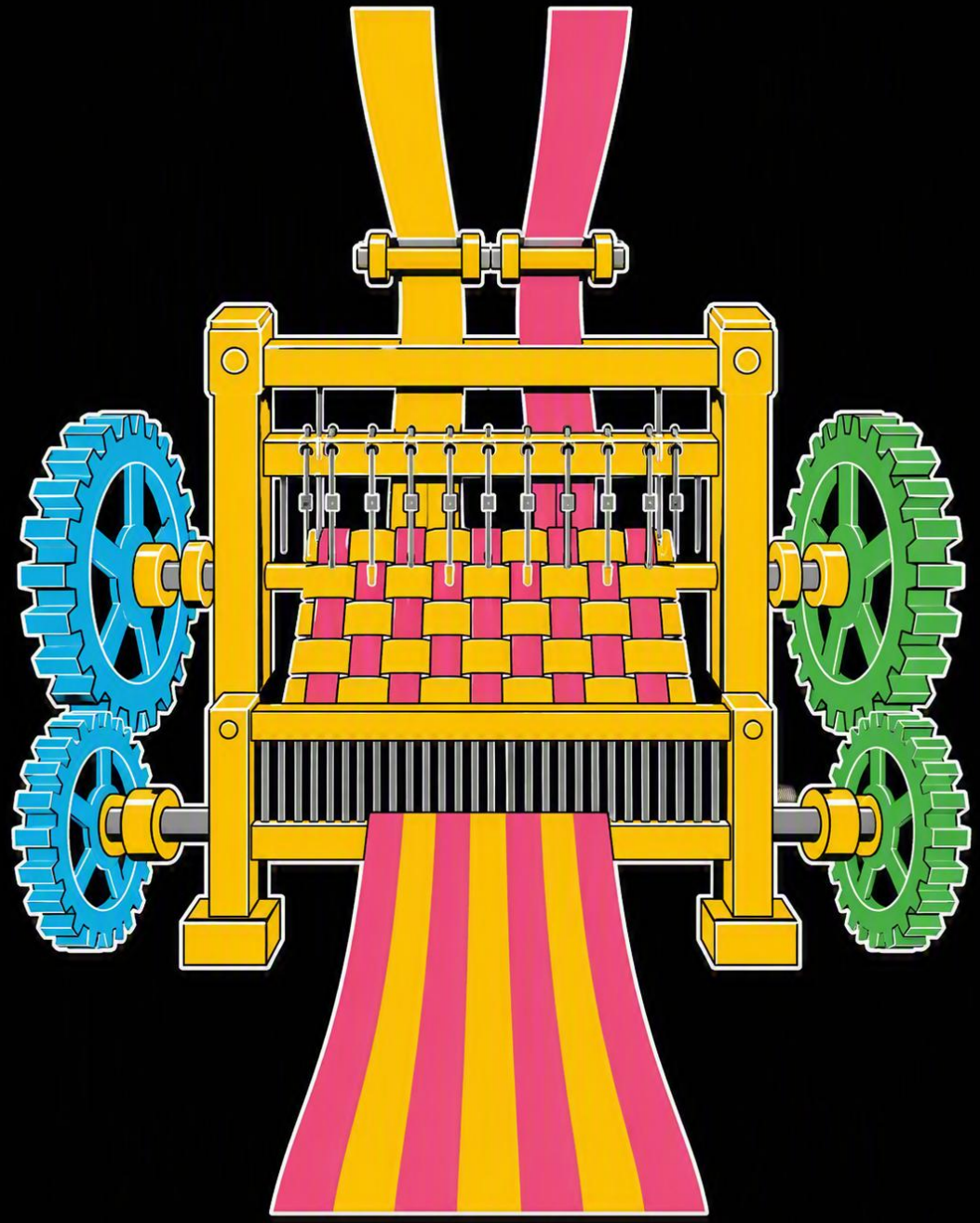
F6 · Drift

Source revoke not applied: open until the next audit

Part 3

The Ugly(?): Policy Weaver bridges the gap

03



Policy Weaver rebuilds source grants as OneLake roles

What?

Supported today

- **Azure Databricks Unity Catalog**
- **Snowflake**
- **Dataverse (beta)**

→

Policy Weaver
reads data security policies from source systems and

→

re-creates them in Microsoft Fabric as **OneLake security roles**

Why?

One source of truth
grants stay in the source system

Automated sync
source roles become OneLake roles on every run

Inventory
one export of all access policies across sources

Fewer manual errors
nobody maintains OneLake roles by hand

How? based on these principles

1. Official Microsoft open-source project: **github.com/microsoft/Policy-Weaver**
2. Pluggable framework
3. Built as a Python library; runs anywhere with a Python runtime, including Fabric notebooks
4. Connectors to source systems evolve independently from how permissions are applied in Fabric
5. Traceability and ability to audit

Policy Weaver in a nutshell

Data and permissions reach OneLake on two separate paths

Source systems

Snowflake Database

Databricks Unity Catalog

More connectors planned

Permissions via Policy Weaver

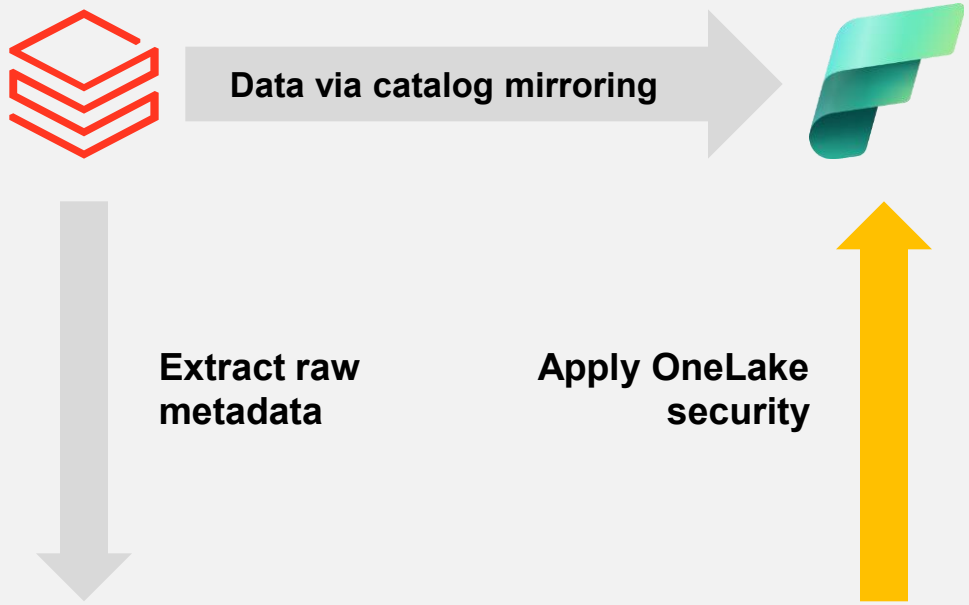
Data via Fabric Mirroring / Shortcuts / Factory

Microsoft Fabric

Mirrored database / Lakehouse

OneLake

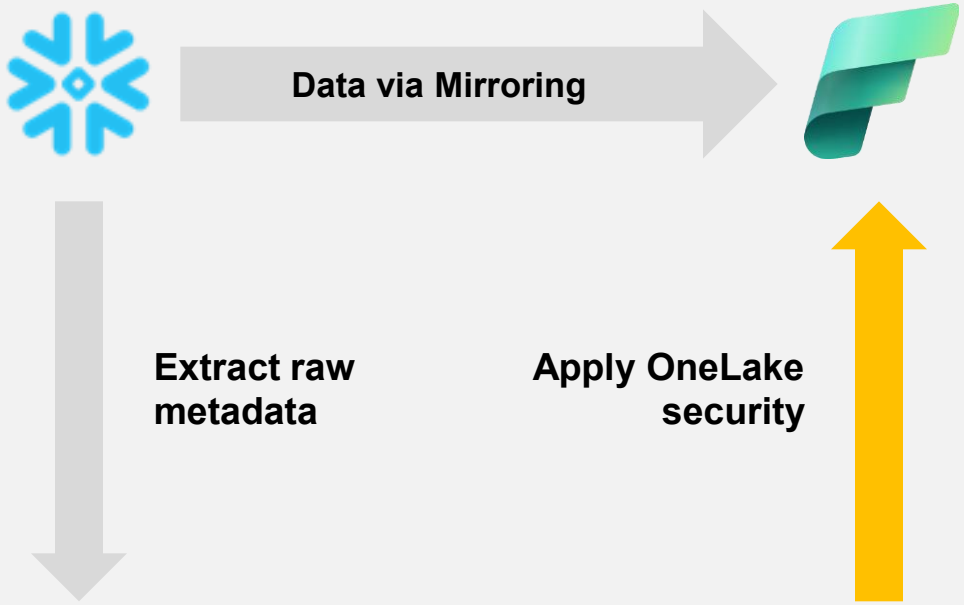
Policy Weaver turns Unity Catalog metadata into OneLake security



What Policy Weaver does

1. **Extract** users, groups and service principals from Unity Catalog (UC) via UC account APIs
2. **Resolve** direct and transitive group memberships
3. **Read** permissions for a defined scope via workspace APIs
4. **Verify** read and usage permissions; resolve all direct and indirect grants
5. **Build** a source-agnostic policy model
6. **Apply** this model to OneLake security

Policy Weaver turns Snowflake account roles into OneLake security



What Policy Weaver does

1. **Extract** users, roles and grants from the SNOWFLAKE.ACCOUNT_USAGE schema
2. **Resolve direct and inherited roles (role hierarchy)**
3. **Verify** read and usage permissions; resolve all direct and indirect grants
4. **Build** a source-agnostic policy model
5. **Apply** this model to OneLake security

Source column masks become column denies in OneLake

Databricks & Snowflake

Support column level security by column mask policies.

OneLake Security

Currently supports column level security only by denying access to specific columns.

Supported column mask policies SQL

```
-- Databricks
CASE WHEN is_account_group_member('HumanResourceDept') THEN ssn ELSE '*****' END
CASE WHEN is_account_group_member('HumanResourceDept') THEN '*****' ELSE ssn END
-- Snowflake
CASE WHEN CURRENT_ROLE() IN ('HR_ROLE') THEN ssn ELSE '-----------' END
CASE WHEN CURRENT_ROLE() IN ('HR_ROLE') THEN '----------' ELSE ssn END
```

Fallback

Configure whether access to the column is granted or denied by default.

All columns denied

If a role is denied every column of a table, the whole table is denied and does not show up in Fabric for that role.

Source filters the table, OneLake filters per role

Databricks, Snowflake and OneLake Security all support row access policies that filter a table on a condition.

Databricks & Snowflake

Apply the policy on the table – NOT the role.

OneLake Security

Applies it on the table IN THE CONTEXT of a role.

Supported row access policy · Snowflake dialect

```
CASE
  WHEN current_role() IN ('SENSITIVE') THEN true
  WHEN current_role() IN ('COUNTRY') THEN Id = 'T001'
  ELSE false
END
```

Equivalent role checks

```
current_role() = 'FINANCE_READER'
current_role() IN ('FINANCE_READER')
```

Fallback

Configure whether access to the table is granted or denied by default.

All rows denied

The whole table is denied to this role and does not show up in Fabric.

All rows allowed

E.g. THEN true – then no row filter is applied.

Four options to tune in config.yaml

```yaml
keyvault:
  use_key_vault: true
  name: policyweaver20250912
  authentication_method: fabric_notebook
fabric:
  mirror_id: 845464654646adfasdf45567
  mirror_name: SFDEMODATA
  workspace_id: 9d556498489465asdf7c
  tenant_id: 3494545asdfs7e2885
  fabric_role_suffix: PW
  delete_default_reader_role: true
  policy_mapping: role_based
constraints:
  columns:
    columnlevelsecurity: true
    fallback: deny
service_principal:
  client_id: kv-service-principal-client-id
  client_secret: kv-service-principal-client-secret
  tenant_id: kv-service-principal-tenant-id
source:
  name: SFDEMODATA
  schemas: <---- optional, if not provided all schemas will be scanned
  - name: analystschema
    tables: <---- optional, if not provided all tables will be scanned
    - subsubanalysttable
type: SNOWFLAKE
databricks:
  workspace_url: https://adb-1441751476278720.0.azuredatabricks.net/
  account_id: 085f281e-a7ef-4faa-9063-325e1db8e45f
  account_api_token: kv-databricks-account-api-token
snowflake:
  account_name: kv-sfaccountname
  user_name: kv-sfusername
  private_key_file: rsa_key.p8
  password: kv-sfpassword
  warehouse: COMPUTE_WH
```

1 **Key Vault integration**

`keyvault: use_key_vault`

2 **Two types of policy mappings in OneLake Security**

`fabric: policy_mapping`

3 **Column-Level Security**

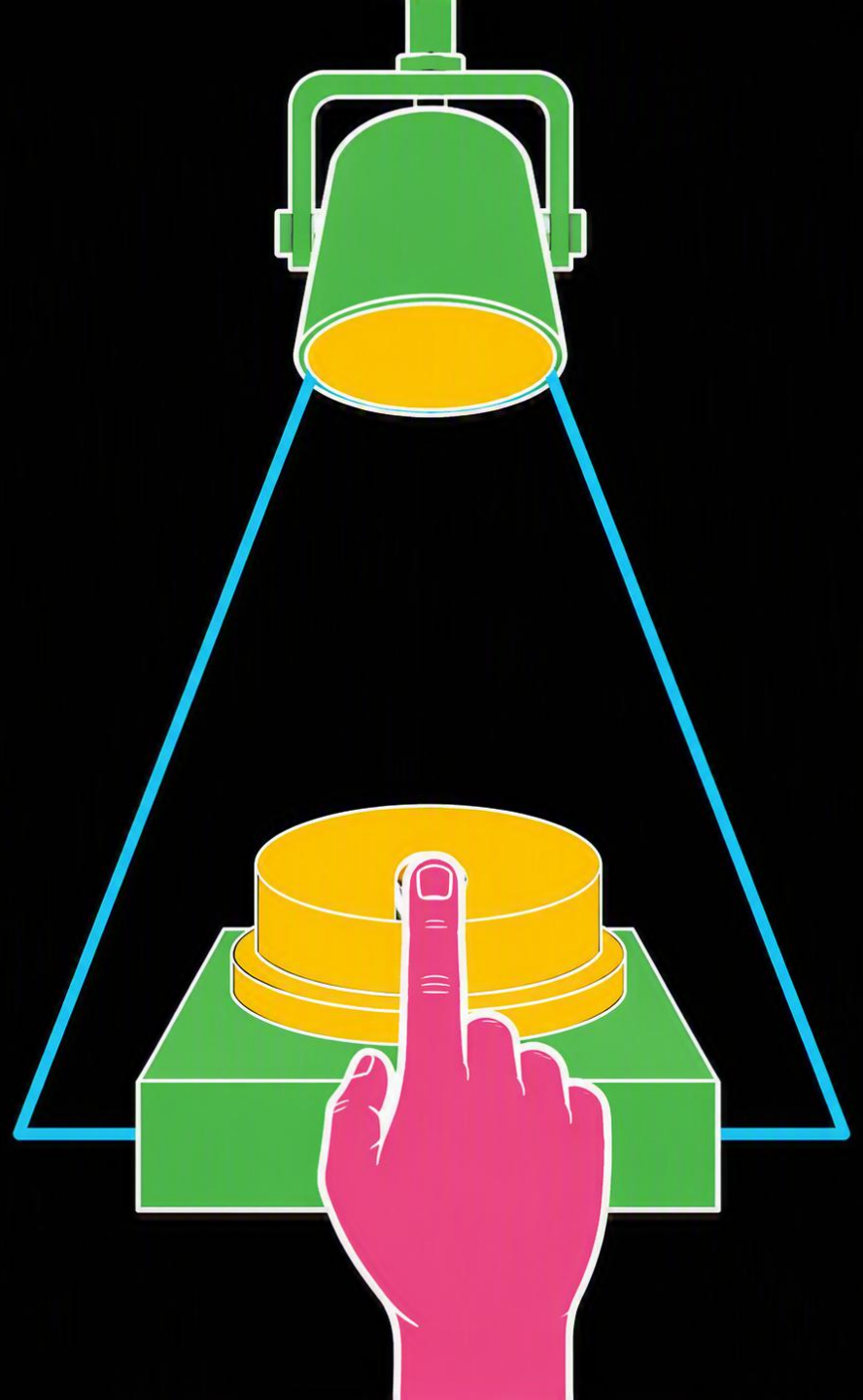
`constraints: columns`

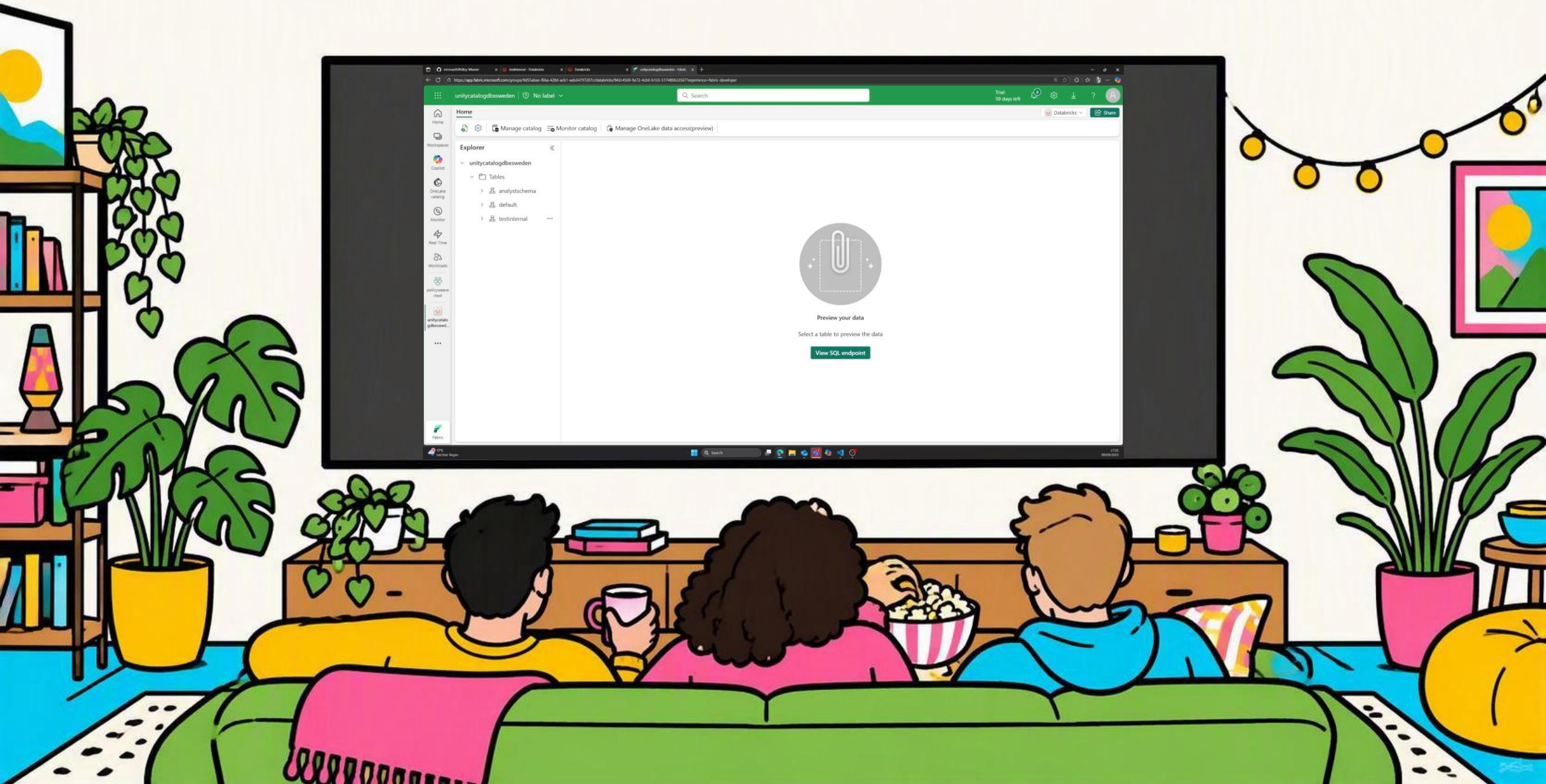
4 **Narrowed scope on schema or table level**

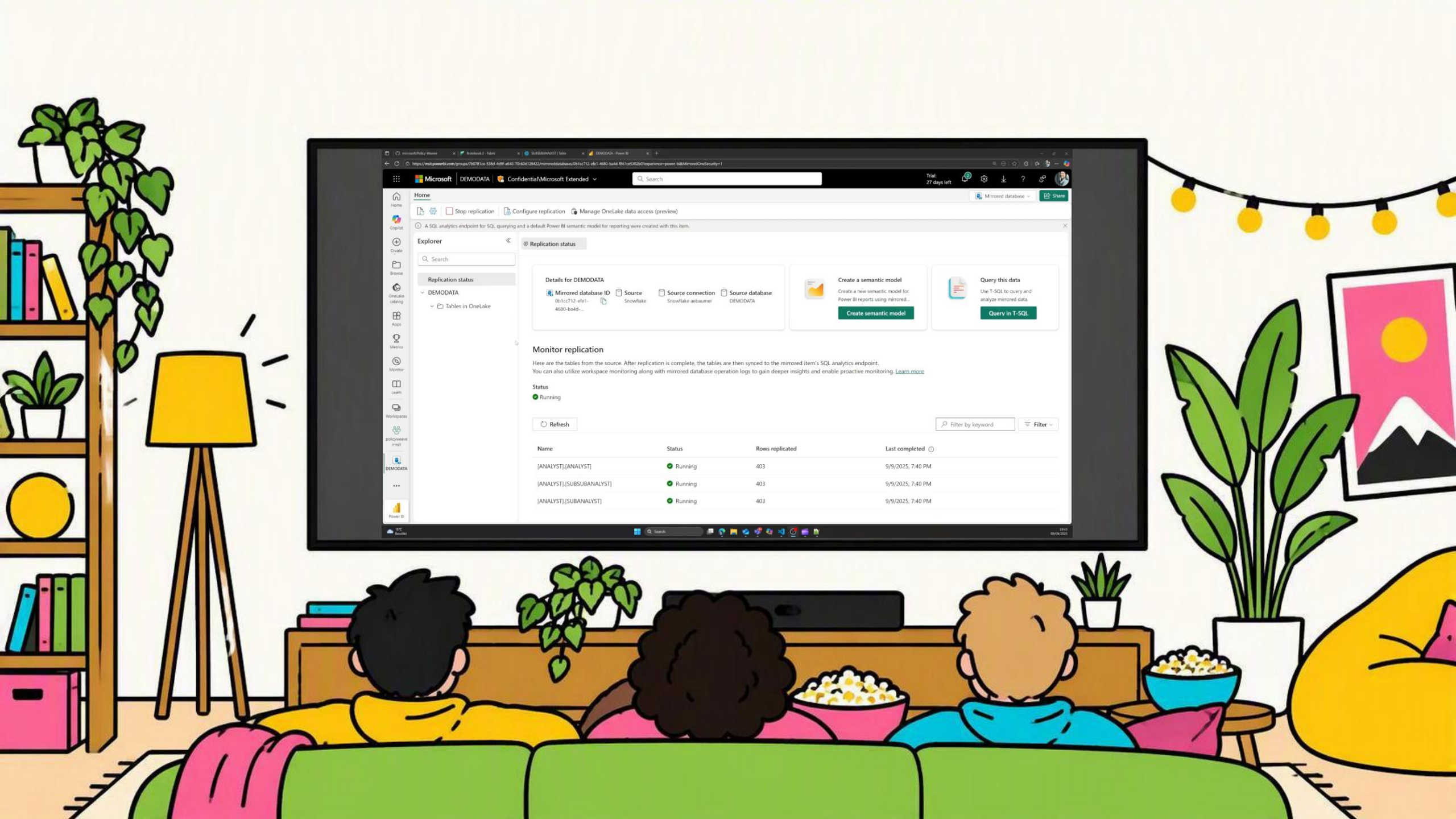
`source: schemas / tables`

Live demo

Live demo: from source grant to verified OneLake role




unitycatalogdbxsweden
No label
Search
Trial:
59 days left
Home
Databricks
Share
Manage catalog
Monitor catalog
Manage OneLake data access(preview)
Explorer
unitycatalogdbxsweden
Tables
analystschema
default
testinternal
Preview your data
Select a table to preview the data
View SQL endpoint


Microsoft
DEMODATA
Confidential\Microsoft Extended
Search
Trial: 27 days left
Home
Mirrored database
Share
Stop replication
Configure replication
Manage OneLake data access (preview)
A SQL analytics endpoint for SQL querying and a default Power BI semantic model for reporting were created with this item.
Explorer
Search
Replication status
DEMODATA
Tables in OneLake
Replication status
Details for DEMODATA
Mirrored database ID
0b1cc712-efe1-4680-ba4d-...
Source
Snowflake
Source connection
Snowflake aebaumer
Source database
DEMODATA
Create a semantic model
Create a new semantic model for Power BI reports using mirrored...
Create semantic model
Query this data
Use T-SQL to query and analyze mirrored data.
Query in T-SQL
Monitor replication
Here are the tables from the source. After replication is complete, the tables are then synced to the mirrored item's SQL analytics endpoint.
You can also utilize workspace monitoring along with mirrored database operation logs to gain deeper insights and enable proactive monitoring. Learn more
Status
Running
Refresh
Filter by keyword
Filter
Name
Status
Rows replicated
Last completed
[ANALYST].[ANALYST]
Running
403
9/9/2025, 7:40 PM
[ANALYST].[SUBSUBANALYST]
Running
403
9/9/2025, 7:40 PM
[ANALYST].[SUBANALYST]
Running
403
9/9/2025, 7:40 PM

Limitations & Assumptions

Identity cleanup, exceptions and monitoring stay with your team

Identity hygiene

Divergent or local identities must be cleaned up first.

Semantic parity

Not every masking or RLS semantic translates 1:1.

Only read permissions

Policy Weaver syncs read access only; write and admin rights need their own model.

Human exceptions

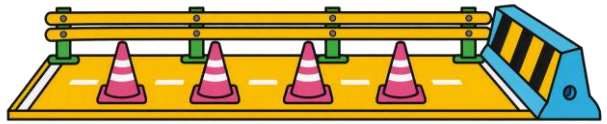
Manually add roles to cover unsupported constructs.

Continuous control

Synchronization must be scheduled, monitored and repeated.

Platform changes

APIs, previews and connectors change; pin versions and retest after updates.



Recommendations

Fabric access matches the source only with clear rules

1 · One source of truth

Grants live in Unity Catalog or Snowflake. Nobody edits OneLake roles by hand.

2 · Deny by default

Set fallback: deny and remove DefaultReader. An unknown policy means no access.

3 · Sync on every change

Run Policy Weaver after each grant change and nightly. Review the diff.

4 · Test as a user

Before go-live, compare what Alice sees in the source and in Fabric. Keep the result.



Policy Weaver copies the rules. Only the user test proves that access is equal.

Native role replication covers Snowflake, Policy Weaver every source

NEW · PREVIEW

Just announced

FabCon Europe 2026: Snowflake Mirroring now replicates roles into OneLake security



	Snowflake Mirroring role replication	Policy Weaver
Status	Public preview since FabCon Europe 2026	Open-source accelerator, no product SLA
Sources	Snowflake only	Azure Databricks Unity Catalog, Snowflake, Dataverse (beta)
Translation scope	Not documented in detail yet: test row access, masking and nested roles	Documented in the README, incl. supported masking policies and RLS/CLS rules
Inventory and audit	Not announced	One export of all access policies across sources
Operation	Runs inside Mirroring, operated by Microsoft	Python library in a Fabric notebook, operated by your team

Decision rule: Snowflake-only estates can test the native path first. Mixed estates with Unity Catalog keep Policy Weaver as the single translation layer.

Status as of October 2026. Source: Microsoft Fabric Updates Blog, FabCon Europe 2026; github.com/microsoft/Policy-Weaver (README).

Mirroring
synchronizes data.

Policy Weaver
synchronizes governance.

Q&A

Matthias Nohl

github.com/nohlimits

linkedin.com/in/matthias-nohl

Matthias.Nohl@btelligent.com

Andreas Rederer

github.com/DaSenf1860

linkedin.com/in/andreas-rederer

andreas.rederer@microsoft.com

youtube.com/@DataAiMonkeyMan

